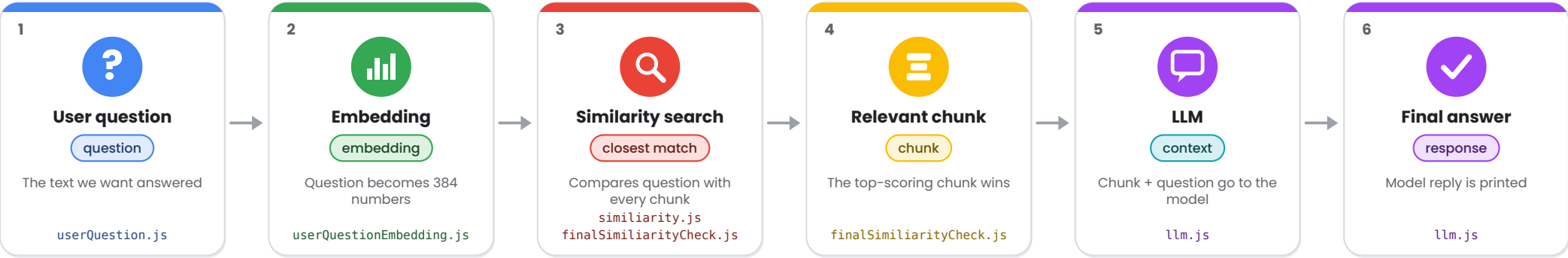


RAG Flow Documentation

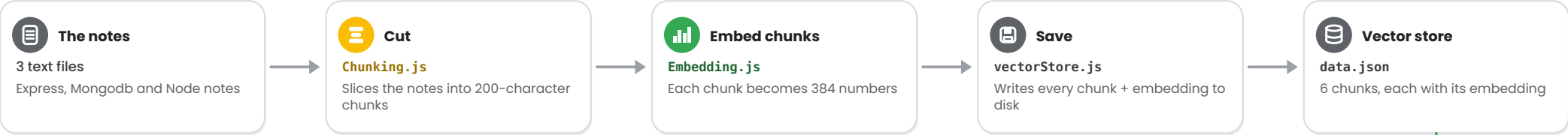
From user question to final answer · explained file by file in simple English

8 JavaScript files · 1 vector store

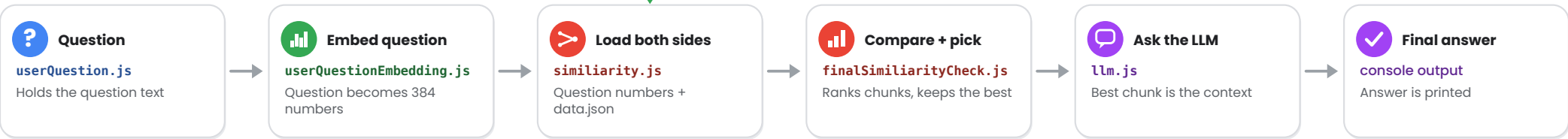
THE COMPLETE FLOW



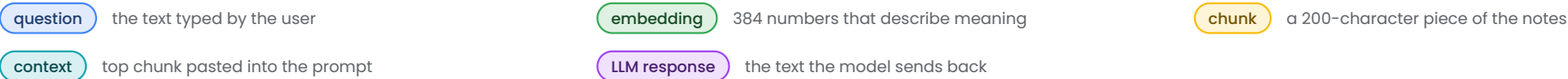
A · PREPARE THE KNOWLEDGE (run once) entry point: vectorStore.js



B · ASK A QUESTION (every run) entry point: llm.js



HIGHLIGHTED DATA



Phase A · Prepare the knowledge

Run once. Turns the text notes into a searchable vector store (data.json).

STEPS 1 - 3

STEP 1



Chunking.js

Cut the notes

PURPOSE

Cuts the three notes into small pieces so each piece can be compared later.

WHAT HAPPENS INSIDE

- Reads Express.txt, Mongodb.txt and Node.txt from ./Documents/
- chunkText() slices each text every 200 characters
- No overlap: a chunk can start or end mid-sentence
- Prints each file's chunks and adds them all to allChunks
- Exports allChunks for Embedding.js

INPUT

Express.txt Mongodb.txt Node.txt

OUTPUT

allChunks

200 characters per chunk

chunks

STEP 2



Embedding.js

Make embeddings

PURPOSE

Turns every chunk into a list of numbers (an embedding) that captures its meaning.

WHAT HAPPENS INSIDE

- Loads the model Xenova/all-MiniLM-L6-v2 (feature-extraction)
- Loops over every chunk from Chunking.js, one at a time
- Runs the model with mean pooling and normalize: true
- Saves { chunk, embedding } - the embedding is 384 numbers
- Prints the total chunks, then returns the list

INPUT

chunks

OUTPUT

{ chunk, embedding } list

384 numbers per chunk

chunk + embedding

STEP 3



vectorStore.js

Save the vector store

PURPOSE

Saves all chunks and their numbers to a file: our tiny vector database.

WHAT HAPPENS INSIDE

- Calls createEmbeddings() from Embedding.js
- Prints the embeddings to the console
- Writes them to data.json with JSON.stringify
- Prints: vector store create successfully
- Runs itself when the file starts: the indexing entry point

INPUT

{ chunk, embedding } list

OUTPUT

data.json

6 entries in the ZIP's data.json



data.json stored once, then read by similarity.js on every question

Phase B · Understand the question

Every run. Get the question, turn it into numbers, and gather what is needed to compare.

STEPS 4 - 6

STEP 4



userQuestion.js

The question

PURPOSE

Holds the question the user wants answered.

WHAT HAPPENS INSIDE

- Stores one string: "what is Node js ?"
- Exports it with module.exports
- No functions and no logic, just the question text

INPUT

typed by hand

OUTPUT

question

used to find the best chunk

question

STEP 5



userQuestionEmbedding.js

Embed the question

PURPOSE

Turns the question into numbers, using the same model as the chunks.

WHAT HAPPENS INSIDE

- Imports question from userQuestion.js
- Loads the same model: Xenova/all-MiniLM-L6-v2
- Runs it with mean pooling and normalize: true
- Converts the output to a plain array with Array.from()
- Prints the question and its embedding, then returns it

INPUT

question

OUTPUT

question embedding

384 numbers

question embedding

STEP 6



similarity.js

Load both sides

PURPOSE

Gathers both sides of the comparison in one place.

WHAT HAPPENS INSIDE

- findSimilarChunk() gets the question embedding first
- Reads and parses data.json (all chunks + embeddings)
- Prints both to the console
- Returns { questionEmbedding, documentEmbedding }
- Despite its name it does no comparing, it only loads

INPUT

question embedding

data.json

OUTPUT

questionEmbedding

documentEmbedding

loads only, no comparing here

continues on the next page: step 7, finalSimilarityCheck.js

Phase B · Find the best chunk and answer

Compare every chunk, keep the closest one, and hand it to the LLM as context.

STEPS 7 - 8

FROM STEP 6

questionEmbedding

documentEmbedding

STEP 7



finalSimilarityCheck.js

Compare and pick

PURPOSE

Checks how close each chunk is to the question and picks the closest one.

WHAT HAPPENS INSIDE

- cosineSimilarity(): dot product / (length of A x length of B)
- Compares every stored chunk with the question embedding
- Builds a list of every chunk with its closeness
- Sorts it from closest to furthest
- Prints and returns only results[0], the top chunk

INPUT

questionEmbedding

documentEmbedding

OUTPUT

chunk

the closest chunk wins

top chunk

STEP 8



llm.js

Ask the LLM

PURPOSE

The file you run to ask a question. It fetches the best chunk, asks the LLM and prints the answer.

WHAT HAPPENS INSIDE

- OpenAI client points to OpenRouter (key: OPENROUTER_API_KEY)
- askQuestion() waits for similarity() to return the top chunk
- Calls model "openrouter/free" with the chunk as system Context
- If context lacks the answer: "I don't know based on the provided documents."
- main() asks "What is Express.js?" and prints Answer:

INPUT

chunk

question

OUTPUT

LLM response

only 1 chunk reaches the LLM

LLM response



Final answer

printed with console.log("Answer:") followed by the model's reply